Lab 02

Lakhvinder Singh

03.03.2026

Lab 02: R as a GIS

Data

The data for this lab can be found in the `./data/CBW/` directory within the course GitHub repository.

Spatial datasets:

1. Streams_Opened_by_Dam_Removal_2012_2017.shp
2. Dam_or_Other_Blockage_Removed_2012_2017.shp
3. County_Boundaries.shp

Non-spatial datasets

1. BMPreport2016_landbmps.csv

Working with tabular data

```
# setup
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.0     v readr     2.1.6
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.1     v tibble    3.3.1
## v lubridate 1.9.4     v tidyr     1.3.2
## v purrr     1.2.1
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(sf)
```

```
## Linking to GEOS 3.13.0, GDAL 3.8.5, PROJ 9.5.1; sf_use_s2() is TRUE
```

```
library(tmap)
```

A "join" is a method to join multiple tables together using a matching "key" found in both datasets. For example:

```
# table of names
t.names <- tibble(key = c(1, 2, 3),
                  name = c("Huey", "Dewey", "Louis"))
```

```r
# table of scores
t.scores <- tibble(name = c("Louis", "Huey", "Dewey"),
                   grade = c(99, 45, 33))

# join them together using a common variable, in this case the "name" variable
t.joined <- left_join(t.names, t.scores, by = "name")
t.joined
```

```
## # A tibble: 3 x 3
##     key name  grade
##   <dbl> <chr> <dbl>
## 1     1 Huey     45
## 2     2 Dewey    33
## 3     3 Louis    99
```

A "left join" finds starts with the table on the "left" and then finds matches in the table on the "right". See the documentation using `?left_join` for more details and for other types of joins. Sometimes the attributes you're using to join the tables won't have the same name, in which case the you need to tell the join function which variables to join on. Note the difference in the `by` argument in the below code block compared to the one above.

```r
t.wonkyNames <- tibble(nombre = c("Dewey", "Louis", "Huey"),
                       x = rep(999),
                       favoriteFood = c("banana", "apple", "carrot"))

t.joined2 <- left_join(t.names, t.wonkyNames, by = c("name" = "nombre"))
t.joined2
```

```
## # A tibble: 3 x 4
##     key name      x favoriteFood
##   <dbl> <chr> <dbl> <chr>
## 1     1 Huey    999 carrot
## 2     2 Dewey   999 banana
## 3     3 Louis   999 apple
```

Let's take a look at some tabular data

This dataset includes a list of best management practices ("BMPs") to reduce nutrient and sediment pollution in the Chesapeake Bay Watershed.

```r
bmps <- read_csv("./CBW/BMPreport2016_landbmps.csv")
```

```
## Rows: 69601 Columns: 18
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (11): StateAbbreviation, GeographyName, Geography, Agency, BMPShortName,...
## dbl  (7): AmountSubmitted, AmountBackedOut, AmountNotBackedOut, AmountCredit...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
glimpse(bmps)
```

```
## Rows: 69,601
## Columns: 18
## $ StateAbbreviation   <chr> "DC", "DC", "DC", "DC", "DC", "DC", "DC", "DC", "D~
## $ GeographyName       <chr> "11001(cbwsonly)", "11001(cbwsonly)", "11001(cbwso~
```

```
## $ Geography           <chr> "Washington, DC (CBWS Portion Only)", "Washington,~
## $ Agency              <chr> "Department of Defense", "Department of Defense", ~
## $ BMPShortName        <chr> "wetpondwetland", "wetpondwetland", "wetpondwetlan~
## $ BMP                 <chr> "Wet Ponds and Wetlands", "Wet Ponds and Wetlands"~
## $ BMPType             <chr> "Efficiency", "Efficiency", "Efficiency", "Efficie~
## $ Unit                <chr> "Acres Treated", "Acres Treated", "Acres Treated",~
## $ Sector              <chr> "Developed", "Developed", "Developed", "Developed"~
## $ FromLoadSource      <chr> "Non-Regulated Roads", "Non-Regulated Buildings an~
## $ ToLoadSource        <chr> "Non-Regulated Roads", "Non-Regulated Buildings an~
## $ AmountSubmitted     <dbl> 8.709123261, 47.984171150, 1.358309392, 6.76626660~
## $ AmountBackedOut     <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ AmountNotBackedOut  <dbl> 8.709123261, 47.984171150, 1.358309392, 6.76626660~
## $ AmountCredited      <dbl> 8.709123261, 47.984171150, 1.358309392, 6.76626660~
## $ Excess              <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ TotalAmountCredited <dbl> 8.709123261, 47.984171150, 1.358309392, 6.76626660~
## $ Cost                <dbl> 11462.077120, 63151.967650, 1787.670991, 8905.0834~
```

Look at the attribute "GeographyName" - it's a character attribute that contains the counties' FIPS code, but also some ancillary explanatory data we need to get rid of. There are multiple ways of doing so, including some (very) fancy automated methods that detect patterns of numbers and characters. We're going to take a simpler approach and assume that all FIPS codes are only 5 characters long.

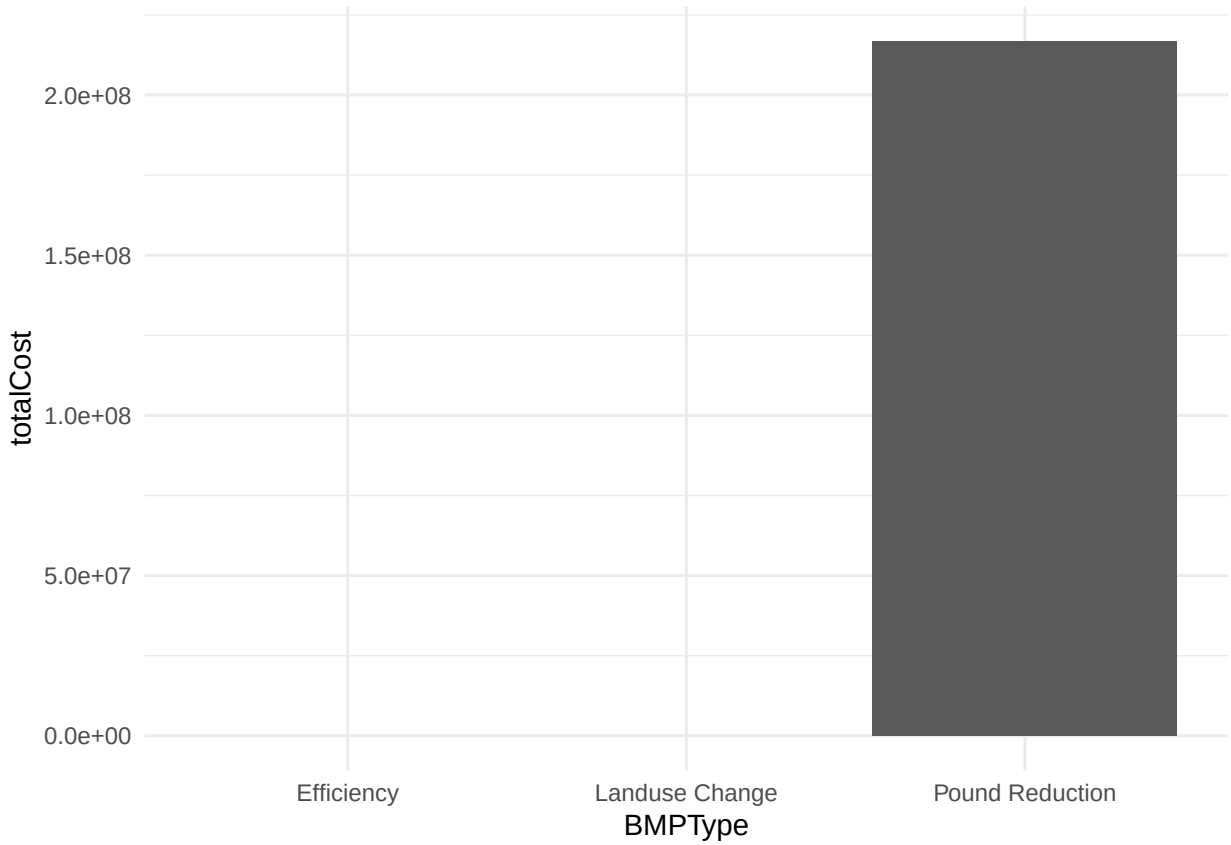
```
# edit the bmps variable in place, which isn't always best practices
bmps <- bmps %>% mutate(., FIPS.trimmed = stringr::str_sub(GeographyName, 1, 5))
```

This preparing raw data for further analysis is called "munging". While the term isn't the important part, these kinds of preparatory steps can be useful when you're trying to create "keys" by which to join tables or just clean your tables in general.

Let's recall how to do some simple tasks

```
# Let's calculate the total cost by BMP and then plot it
bmps %>% group_by(BMPType) %>% summarise(totalCost = sum(Cost)) %>%
  ggplot(., aes(x = BMPType, y = totalCost)) +
  geom_bar(stat = "identity") +
  theme_minimal()
```

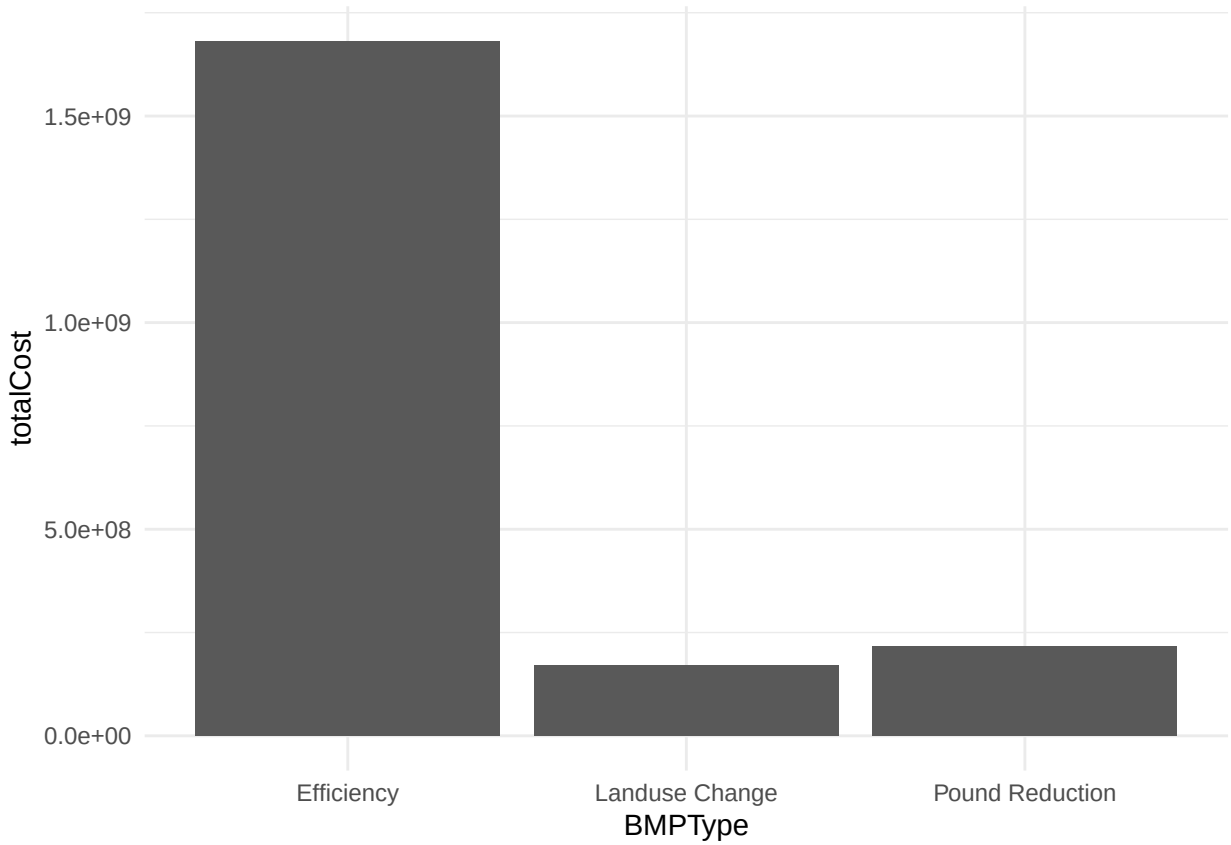
```
## Warning: Removed 2 rows containing missing values or values outside the scale range
## (`geom_bar()`).
```




```
# Doesn't really work. This is because there are missing data in the cost attribute.
# Let's look at it (and this is why we do exploratory data analysis first)

summary(bmps$Cost)
```

```
##     Min.  1st Qu.   Median     Mean  3rd Qu.     Max.     NA's
##        0        0      142    37408     3583 39731974    14286
```

```
# Yup, lots of "NA's"... We can drop them in our analysis.
# Look carefully at the code in the 'sum' function. What is the na.rm = T parameter doing?


bmps %>% group_by(BMPType) %>% summarise(totalCost = sum(Cost, na.rm = T)) %>%
  ggplot(., aes(x = BMPType, y = totalCost)) +
  geom_bar(stat = "identity") +
  theme_minimal()
```



We can also group by multiple variables at the same time. For example:

```
# group by state and sector, sum total cost
twofactors <- bmps %>% group_by(StateAbbreviation, Sector) %>%
  summarise(totalCost = sum(Cost))
```

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by StateAbbreviation and Sector.
## i Output is grouped by StateAbbreviation.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(StateAbbreviation, Sector))` for per-operation
##   grouping (`?dplyr::dplyr_by`) instead.
```

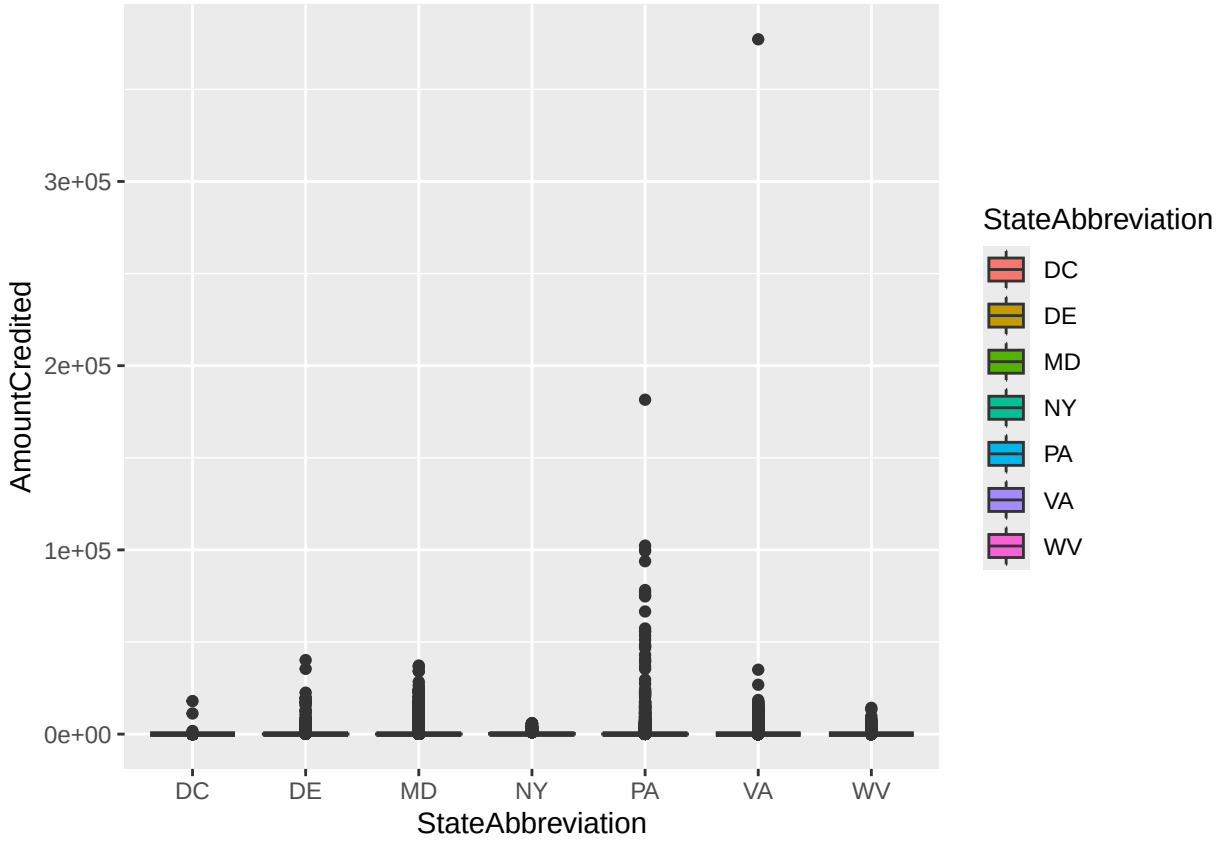
```
twofactors
```

```
## # A tibble: 25 x 3
## # Groups:   StateAbbreviation [7]
##    StateAbbreviation Sector      totalCost
##    <chr>             <chr>           <dbl>
##  1 DC                Developed          NA
##  2 DC                Natural            NA
##  3 DE                Agriculture        NA
##  4 DE                Developed          NA
##  5 DE                Natural            NA
##  6 DE                Septic       2064477.
##  7 MD                Agriculture        NA
##  8 MD                Developed          NA
##  9 MD                Natural            NA
## 10 MD                Septic      50470281.
```

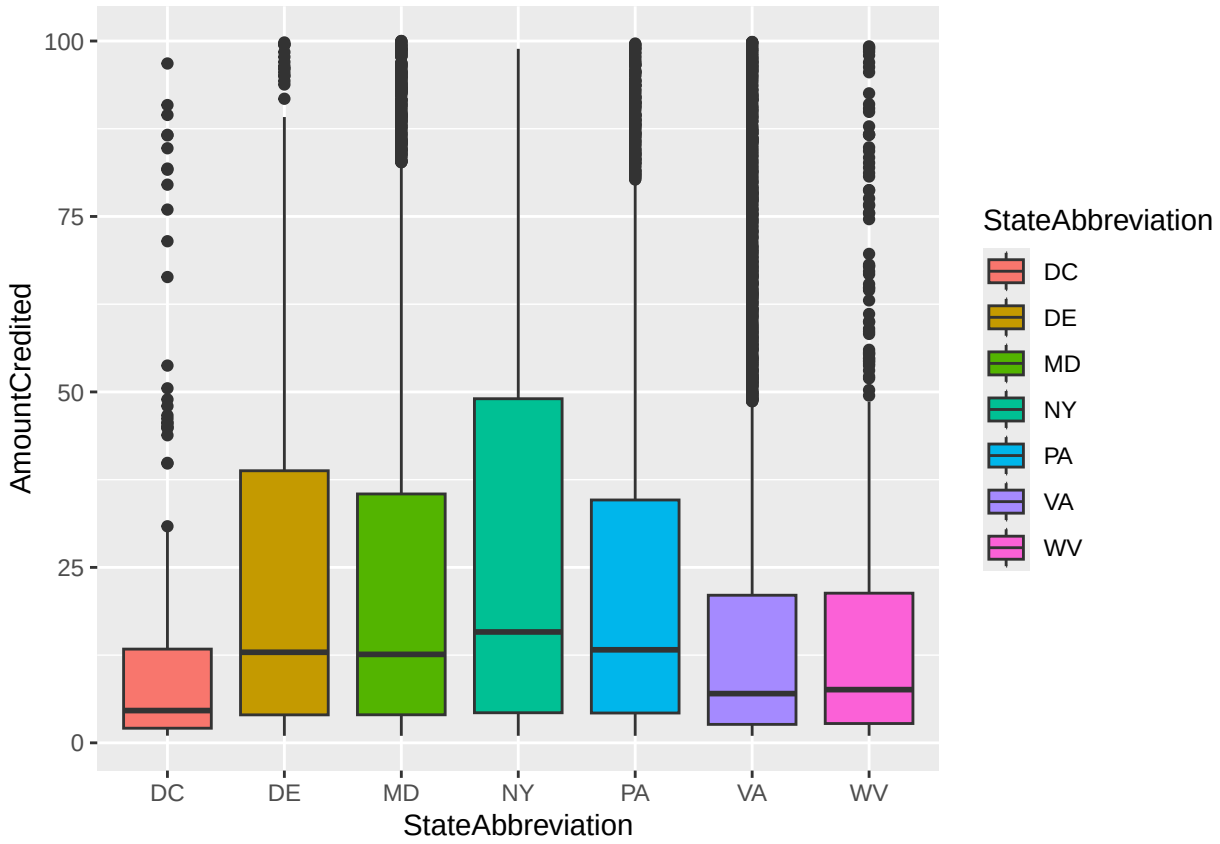
```
## # i 15 more rows
```

For our last bit of review, let's make a few box plots:

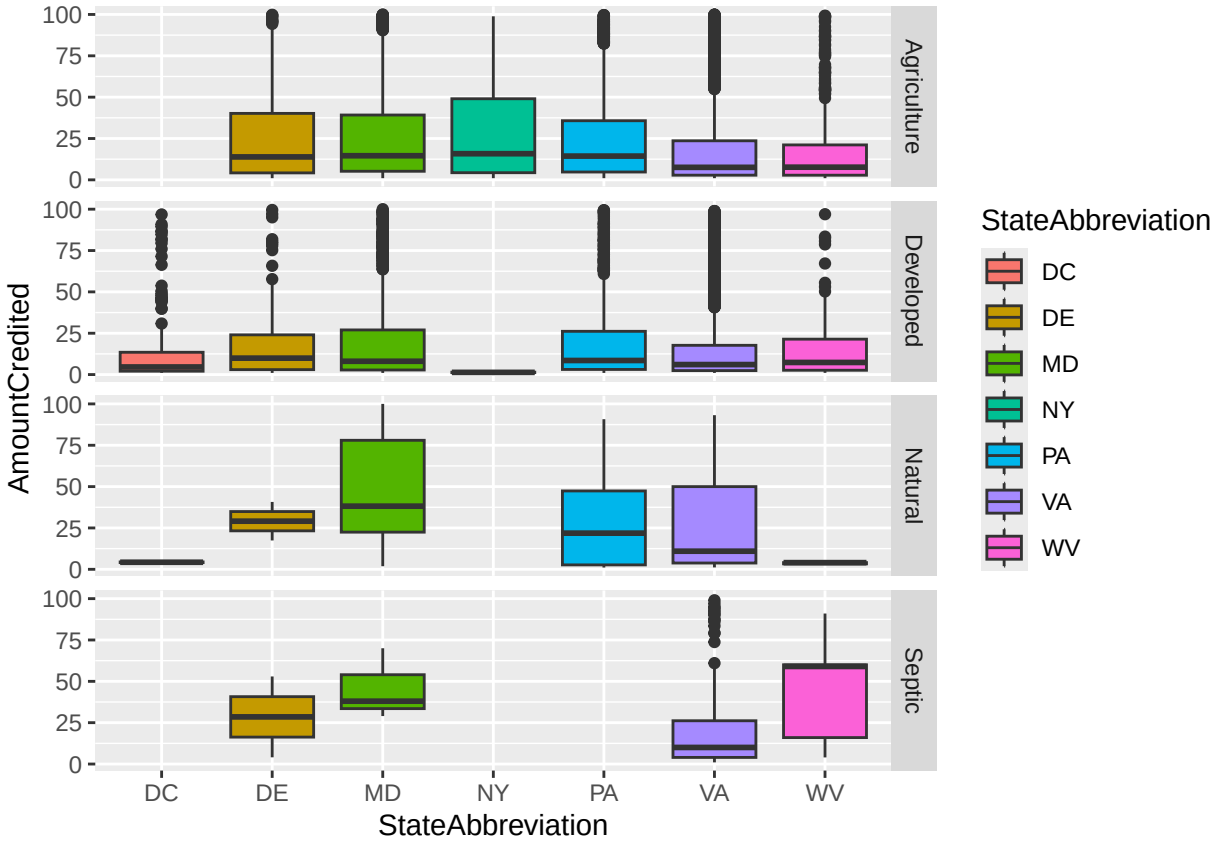
```
# A simple one
bmps %>% ggplot(., aes(x = StateAbbreviation, y = AmountCredited)) +
  geom_boxplot(aes(fill = StateAbbreviation))
```




```
# Very heavily skewed, so just for the sake of visualization,
# let's subset the data (dramatically)
bmps %>%
  dplyr::filter(., AmountCredited > 1 & AmountCredited < 100) %>%
  ggplot(., aes(x = StateAbbreviation, y = AmountCredited)) +
  geom_boxplot(aes(fill = StateAbbreviation))
```



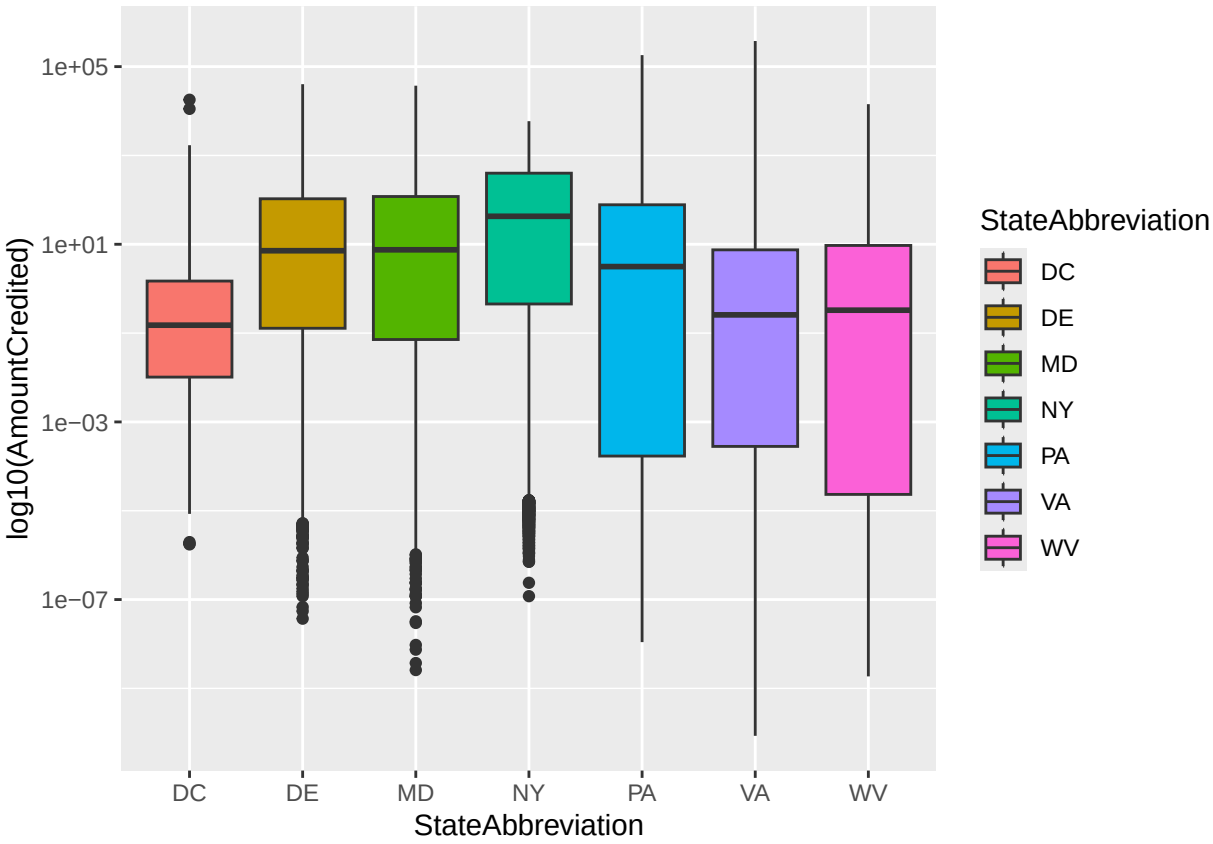

```
# We can also plot multiple dimensions in our plot using
# the `facet` family of commands in ggplot

bmps %>%
  dplyr::filter(., AmountCredited > 1 & AmountCredited < 100) %>%
  ggplot(., aes(x = StateAbbreviation, y = AmountCredited)) +
  geom_boxplot(aes(fill = StateAbbreviation)) +
  facet_grid(Sector~.)
```




```
# finally, an alternative to removing the outliers would be to log-transform the y-axis
# But be careful, this removes a lot of the data because the log of 0 is undefined
bmps %>% ggplot(., aes(x = StateAbbreviation, y = AmountCredited)) +
  geom_boxplot(aes(fill = StateAbbreviation)) +
  scale_y_log10() +
  labs(y = "log10(AmountCredited)")
```

```
## Warning in scale_y_log10(): log-10 transformation introduced infinite values.
```

```
## Warning: Removed 15585 rows containing non-finite outside the scale range
## (`stat_boxplot()`).
```



The last new item uses the `%in%` command. It's a way to quickly figure out which elements are inside of another. In that sense, it's similar to a spatial intersection, but for other types of data.

```
# setup a demonstration vector
x <- c(1, 2, 3, 4, 5)

# is 7 in our vector?
7 %in% x # should evaluate False
```

```
## [1] FALSE
```

```
2 %in% x # should evaluate True
```

```
## [1] TRUE
```

```
# can also do it with vectors
c(4, 99, 1) %in% x
```

```
## [1]  TRUE FALSE  TRUE
```

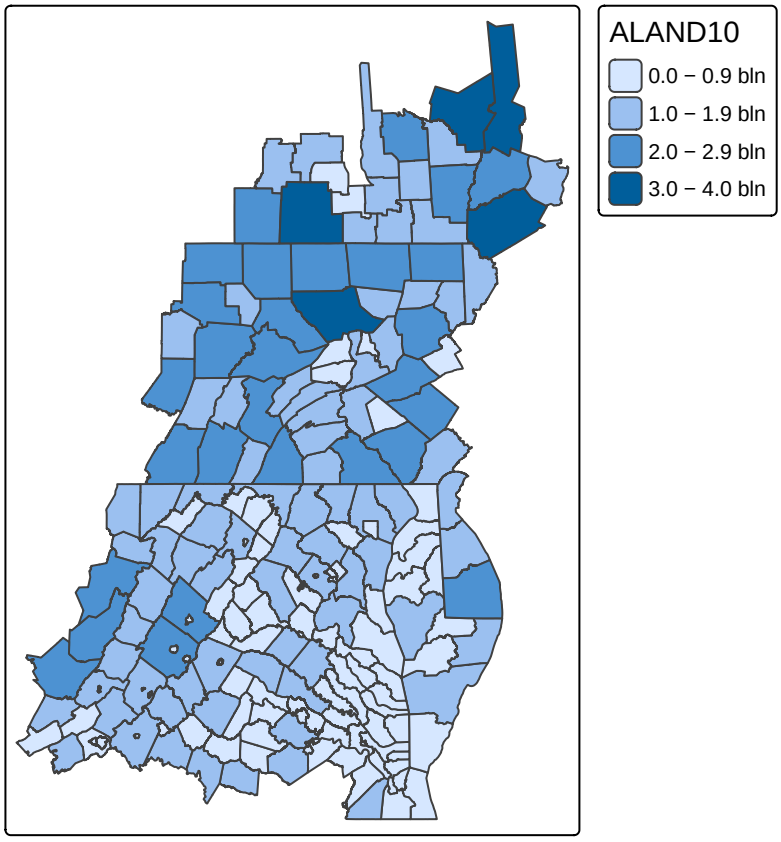
Lastly, let's recall using tmap on our data. Also remember you can use `sf::st_make_valid` to fix offending geometry

```
counties <- sf::read_sf("./CBW/County_Boundaries.shp")
counties %>% sf::st_is_valid()
```

```
##   [1]  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
##  [13]  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
##  [25]  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
##  [37]  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
##  [49]  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
```

```
##  [61]  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
##  [73]  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
##  [85]  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
##  [97]  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
## [109]  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
## [121]  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
## [133]  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
## [145]  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
## [157]  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
## [169]  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
## [181]  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
## [193]  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
## [205]  TRUE FALSE  TRUE
```

```
counties <- counties %>% sf::st_make_valid()

# quick map of the data
tm_shape(counties) + tm_polygons(fill = "ALAND10")
```



Your tasks

Using the following data...

```
# spatial
# Note, the st_make_valid() calls are NOT necessary in all cases
# But we know it's needed for the counties, and doesn't really hurt anything
# for the others
counties <- sf::read_sf("./CBW/County_Boundaries.shp") %>%
  sf::st_make_valid()
dams <- sf::read_sf("./CBW/Dam_or_Other_Blockage_Removed_2012_2017.shp") %>%
  sf::st_make_valid()
streams <- sf::read_sf("./CBW/Streams_Opened_by_Dam_Removal_2012_2017.shp") %>%
  sf::st_make_valid()

# aspatial
bmps <- read_csv("./CBW/BMPreport2016_landbmps.csv")
```

```
## Rows: 69601 Columns: 18
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (11): StateAbbreviation, GeographyName, Geography, Agency, BMPShortName,...
## dbl  (7): AmountSubmitted, AmountBackedOut, AmountNotBackedOut, AmountCredit...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

...complete the following tasks.

Complete each task COMPLETELY USING R CODE. YOU MUST SHOW YOUR WORK FOR EACH ANSWER. Label your variables sensibly and use comments such that I can find your answers and your work. The following tasks draw upon lecture, your assigned readings, and the examples shown above. As always, there are multiple ways of completing each task. Remember, it's always a good idea to perform some exploratory data analysis on your own prior to starting work.

Task 1: Aspatial operations

```
#1.1 Calculate summary statistics for the Cost of BMPs for each State (including DC)
# na.rm = True removes missing values so math works. Cheers.
bmp_cost_summary <- bmps %>%
  group_by(StateAbbreviation) %>%
  summarise(
    mean_cost = mean(Cost, na.rm = TRUE),
    median_cost = median(Cost, na.rm = TRUE),
    sd_cost = sd(Cost, na.rm = TRUE),
    min_cost = min(Cost, na.rm = TRUE),
    max_cost = max(Cost, na.rm = TRUE)
  )

bmp_cost_summary
```

```
## # A tibble: 7 x 6
##   StateAbbreviation mean_cost median_cost sd_cost min_cost  max_cost
##   <chr>                <dbl>       <dbl>   <dbl>    <dbl>     <dbl>
## 1 DC                  22193.       629.  106690.   0.0510  1891064.
## 2 DE                  30682.       906.  173938.   0       5041838.
```

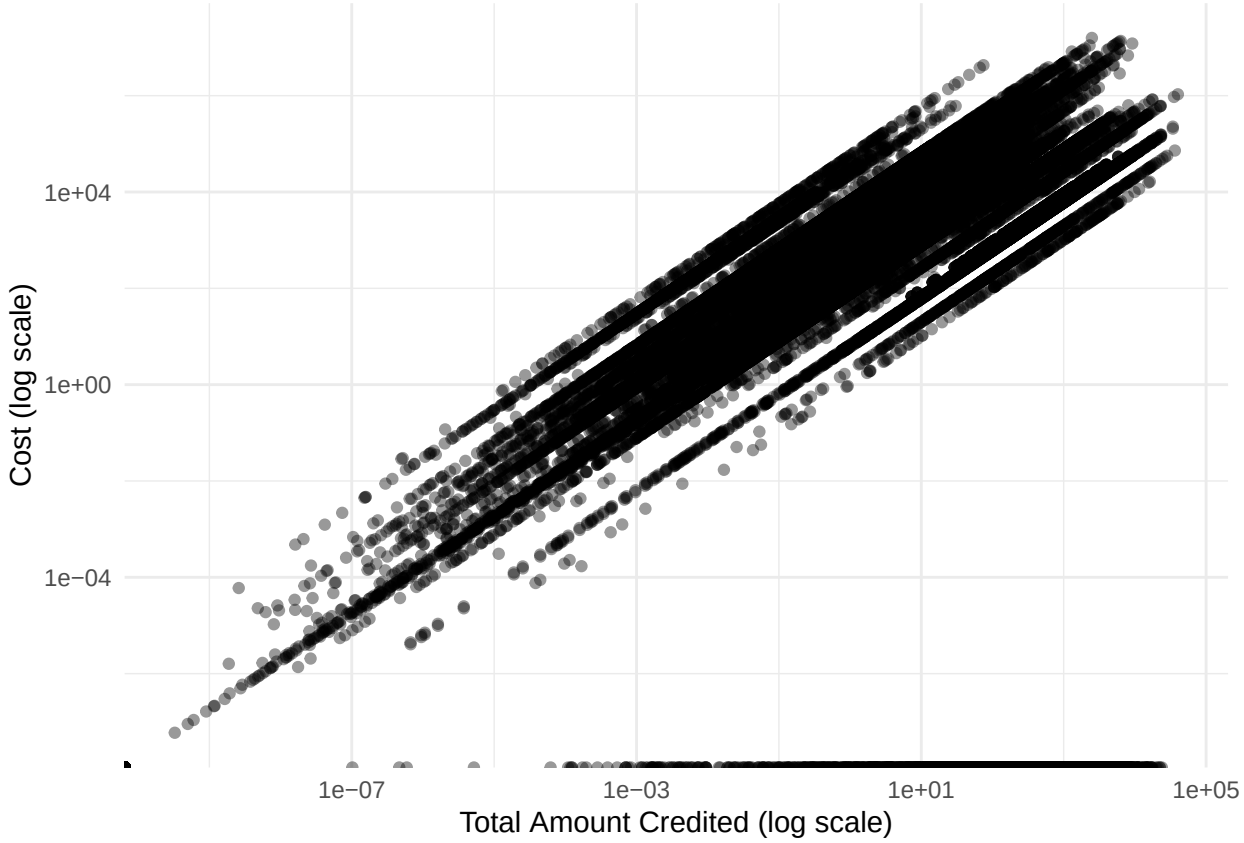
```
## 3 MD                  53118.       876.  356888.    0      12086857.
## 4 NY                   8129.      1954.   27789.    0        622138.
## 5 PA                  59904.       431.  473775.    0      19968186
## 6 VA                  25380.        15.2 348963.    0      39731974.
## 7 WV                  50014.        96.8 521885.    0      15796521.
```

```r
#1.2 Make a scatterplot of Cost vs. TotalAmountCredited, ONLY FOR BMPs with `Units` of type "Acres". Yo

# Need to Keep only BMPs measured in Acres, also we are applying log scales to reduce the effect of ext
bmps %>%
  filter(str_detect(Unit, "Acres")) %>%
  ggplot(aes(x = TotalAmountCredited, y = Cost)) +
  geom_point(alpha = 0.4) +
  scale_x_log10() +
  scale_y_log10() +
  labs(
    x = "Total Amount Credited (log scale)",
    y = "Cost (log scale)"
  ) +
  theme_minimal()
```

```
## Warning in scale_x_log10(): log-10 transformation introduced infinite values.
```

```
## Warning in scale_y_log10(): log-10 transformation introduced infinite values.
```

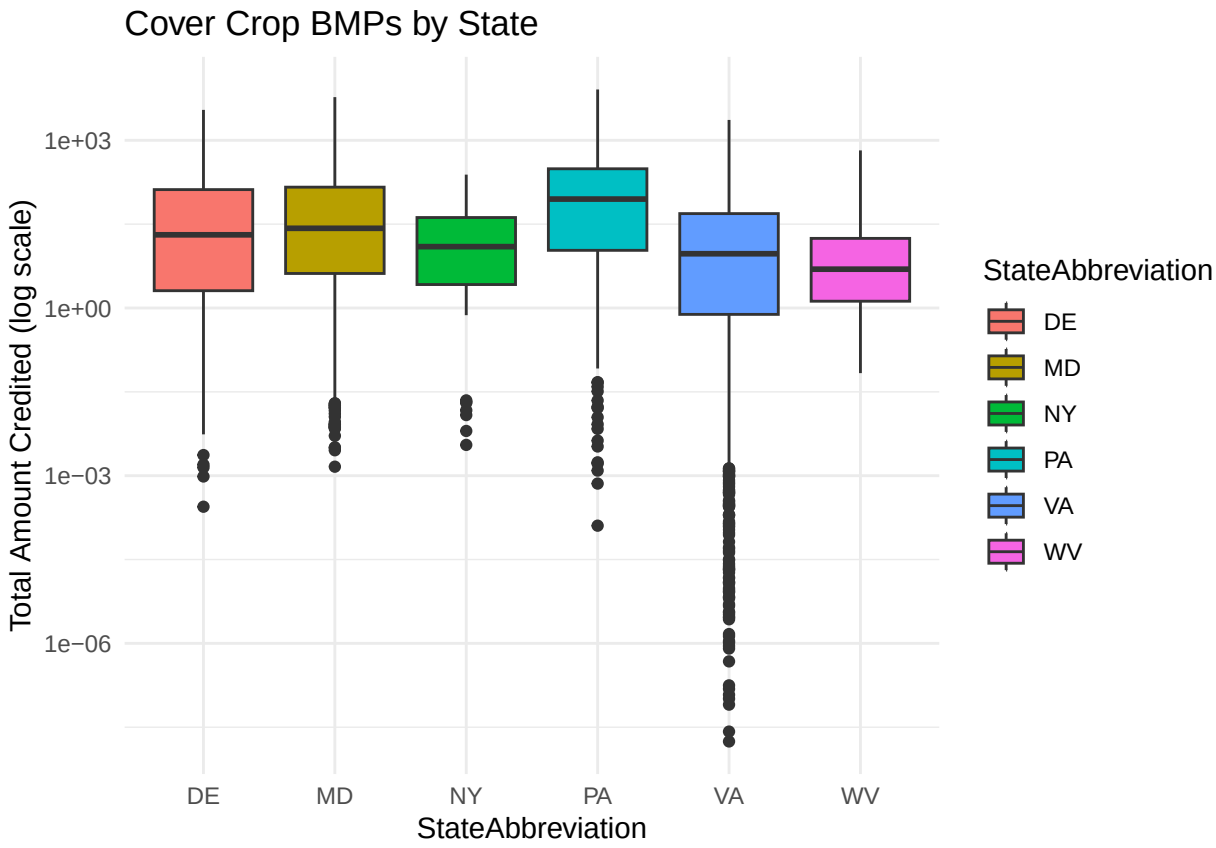
```
## Warning: Removed 14286 rows containing missing values or values outside the scale range
## (`geom_point()`).
```



```r
#1.3 Make a boxplot with "StateAbbreviation" on the x-axis and "TotalAmountCredited" on the y-axis. HOW

# str_detect() can be applied to find all BMPS that include "COVER WORD"
cover_crop_bmps <- bmps %>%
  filter(str_detect(str_to_lower(BMP), "cover"))

# Boxplot by state
ggplot(cover_crop_bmps,
       aes(x = StateAbbreviation, y = TotalAmountCredited)) +
  geom_boxplot(aes(fill = StateAbbreviation)) +
  scale_y_log10() +
  labs(
    y = "Total Amount Credited (log scale)",
    title = "Cover Crop BMPs by State"
  ) +
  theme_minimal()
```

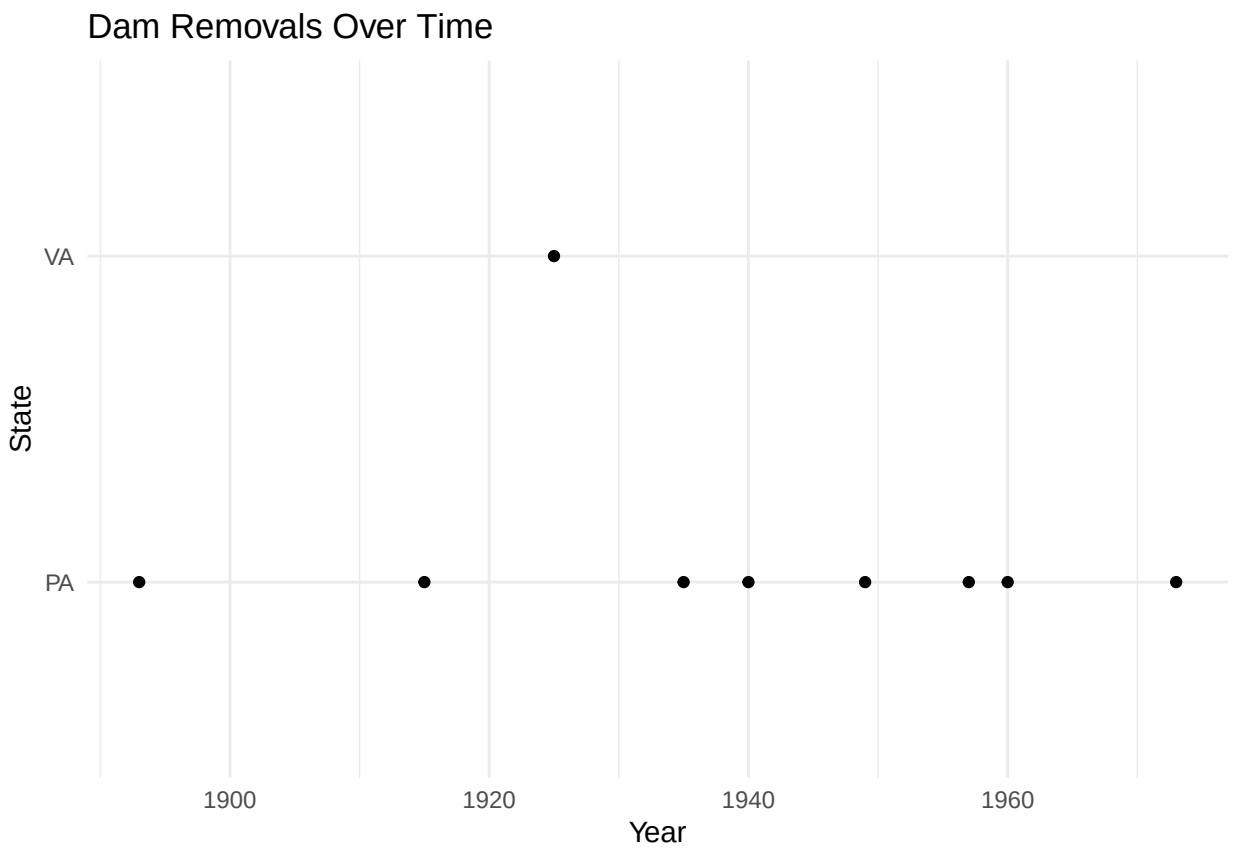
```
## Warning in scale_y_log10(): log-10 transformation introduced infinite values.
```

```
## Warning: Removed 427 rows containing non-finite outside the scale range
## (`stat_boxplot()`).
```




```r
#1.4 make a scatterplot of the `dam` dataset, this time with "YEAR" on the x-axis and "STATE" on y-axis

# Removed data points assuming no dams built before year 0 by using filter (YEAR > 0)
dams %>%
  filter(YEAR > 0) %>%
  ggplot(aes(x = YEAR, y = STATE)) +
```

```
  geom_point() +
  labs(
    x = "Year",
    y = "State",
    title = "Dam Removals Over Time"
  ) +
  theme_minimal()
```

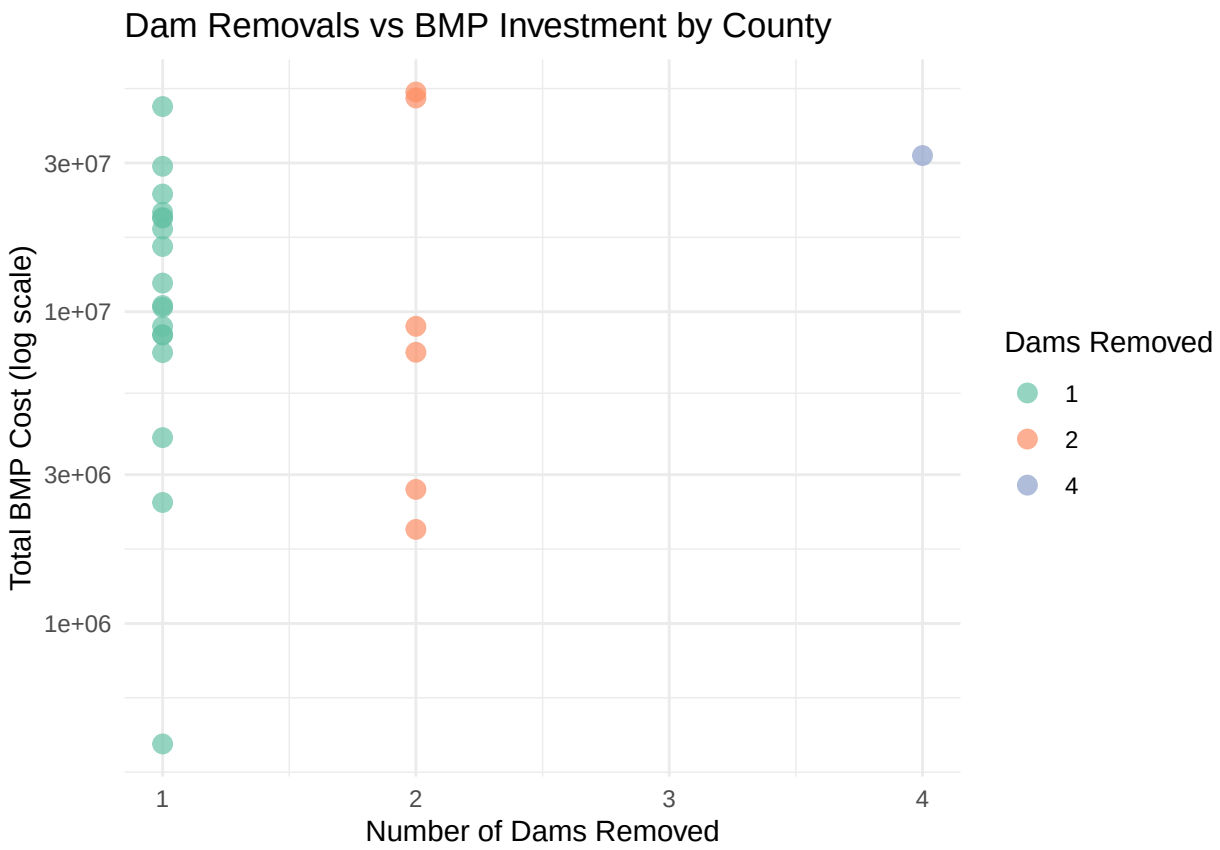



```
#1.5 make one last (aspatial) visualization. But this time, it's your choice what data and plots to use

library(tidyverse)
library(sf)

# Read data
counties <- read_sf("./CBW/County_Boundaries.shp") %>%
  st_make_valid()

dams <- read_sf("./CBW/Dam_or_Other_Blockage_Removed_2012_2017.shp") %>%
  st_make_valid()

bmps <- read_csv("./CBW/BMPreport2016_landbmps.csv") %>%
  mutate(FIPS = str_sub(GeographyName, 1, 5))
```

```
## Rows: 69601 Columns: 18
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (11): StateAbbreviation, GeographyName, Geography, Agency, BMPShortName,...
```

```
## dbl  (7): AmountSubmitted, AmountBackedOut, AmountNotBackedOut, AmountCredit...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
# Count dams per county (USE GEOID10) , dam_n means no of dams removed in a county
dams_cty <- st_join(dams, counties) %>%
  group_by(GEOID10) %>%
  summarise(dam_n = n(), .groups = "drop")

# Sum BMP cost per county (USE FIPS)
bmp_cty <- bmps %>%
  group_by(FIPS) %>%
  summarise(bmp_cost = sum(Cost, na.rm = TRUE))

# Joining both summaries
cty_join <- left_join(dams_cty, bmp_cty,
                      by = c("GEOID10" = "FIPS"))

# Scatter plot
ggplot(cty_join, aes(dam_n, bmp_cost, color = factor(dam_n))) +
  geom_point(size = 3, alpha = 0.7) +
  scale_color_brewer(palette = "Set2") +
  scale_y_log10() +
  labs(
    x = "Number of Dams Removed",
    y = "Total BMP Cost (log scale)",
    color = "Dams Removed",
    title = "Dam Removals vs BMP Investment by County"
  ) +
  theme_minimal()
```



Task 2: Spatial operations

```
#2.1 Find the 5 longest streams in the 'streams opened by dam removal' dataset

# Finding 5 longest streams

streams_long <- streams %>%
  mutate(length_m = as.numeric(st_length(.))) %>%
  arrange(desc(length_m))

streams_long %>%
  slice(1:5)
```

```
## Simple feature collection with 5 features and 90 fields
## Geometry type: LINESTRING
## Dimension:     XY
## Bounding box:  xmin: -79.06165 ymin: 37.96144 xmax: -76.05378 ymax: 41.66711
## Geodetic CRS:  WGS 84
## # A tibble: 5 x 91
##   OBJECTID_1   ComID Permanent_ FDate      Resolution GNIS_ID GNIS_Name LengthKM
##        <int>   <int> <chr>      <date>     <chr>      <chr>   <chr>        <dbl>
## 1       7470  5.75e7 <NA>       1970-01-01 <NA>       <NA>    Little C~     6.85
## 2       6693  4.14e7 <NA>       1970-01-01 <NA>       <NA>    Johns Run     5.43
## 3       7794  6.58e7 <NA>       1970-01-01 <NA>       <NA>    Little T~     5.07
## 4       7848  6.58e7 <NA>       1970-01-01 <NA>       <NA>    Great Tr~     4.89
## 5       8469  6.64e7 <NA>       1970-01-01 <NA>       <NA>    Little M~     4.43
## # i 83 more variables: ReachCode <chr>, FlowDir <chr>, WBAreaComI <chr>,
```

```
## #   WBArea_Per <chr>, FType <int>, FCode <int>, Enabled <chr>, SourceHUC <chr>,
## #   comments <chr>, to_steward <chr>, DeleteMe <chr>, Edits <chr>,
## #   Bifurc <chr>, CtchSqMi <chr>, BayWater <chr>, HYDROID <int>,
## #   FROM_NODE <int>, TO_NODE <int>, NextDownID <int>, DA_SqMi <chr>,
## #   NESZCL <chr>, strahler <int>, shreve <int>, Above10per <chr>,
## #   Shape_Leng <chr>, Region <chr>, Fnode <chr>, Tnode <chr>, ...
```

```
#2.2 Find the three counties with the greatest TOTAL length of streams (opened by dam removal) in them

# Assign each stream to a county
streams_cty <- st_join(streams, counties)

# Sum total stream length per county
streams_summary <- streams_cty %>%
  mutate(length_m = st_length(.)) %>%
  group_by(NAME10) %>%                    # county name column
  summarise(total_length = sum(length_m),
            .groups = "drop") %>%
  arrange(desc(total_length))

# Top 3 counties
streams_summary %>% slice(1:3)
```

```
## Simple feature collection with 3 features and 2 fields
## Geometry type: MULTILINESTRING
## Dimension:     XY
## Bounding box:  xmin: -79.23801 ymin: 37.88782 xmax: -77.96634 ymax: 41.62481
## Geodetic CRS:  WGS 84
## # A tibble: 3 x 3
##   NAME10     total_length                                              geometry
##   <chr>             [m]                                   <MULTILINESTRING [°]>
## 1 Augusta         424897. ((-79.20884 37.98276, -79.20882 37.98262, -79.20881 3~
## 2 Huntingdon      308558. ((-78.01929 40.36854, -78.01908 40.36834, -78.01875 4~
## 3 Cameron         241557. ((-78.40534 41.57611, -78.40511 41.57597, -78.40495 4~
```

```
#2.3 Make a map of the counties, shading each county by the total cost of BMPs funded/implemented in th

bmps <- read_csv("./CBW/BMPreport2016_landbmps.csv") %>%
  mutate(FIPS = str_sub(GeographyName, 1, 5))
```

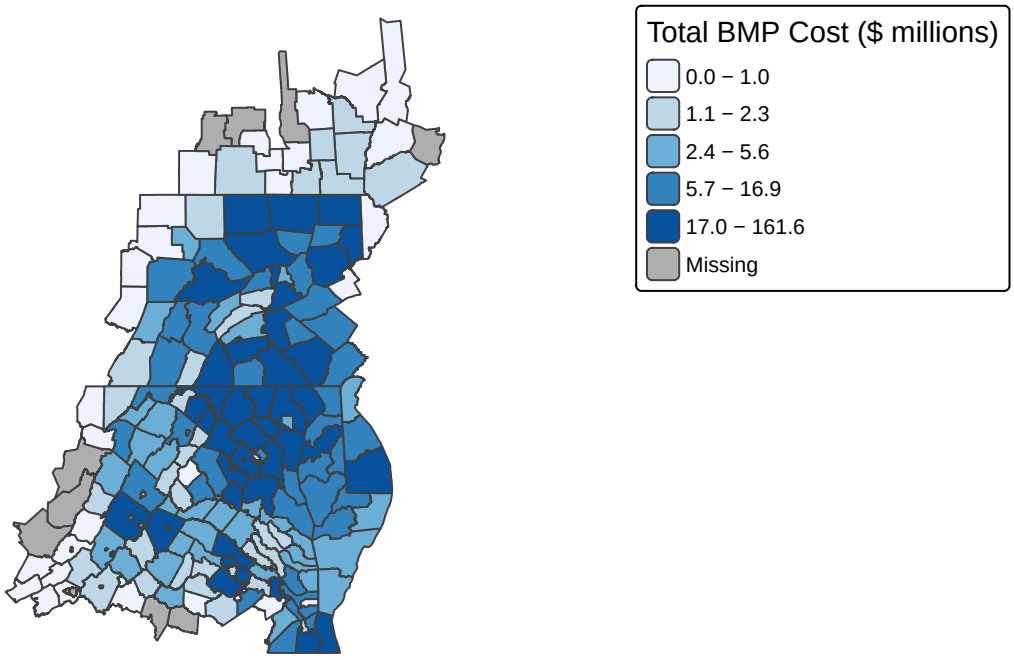
```
## Rows: 69601 Columns: 18
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (11): StateAbbreviation, GeographyName, Geography, Agency, BMPShortName,...
## dbl  (7): AmountSubmitted, AmountBackedOut, AmountNotBackedOut, AmountCredit...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# Sum BMP cost per county
bmp_cty <- bmps %>%
  group_by(FIPS) %>%
  summarise(total_cost = sum(Cost, na.rm = TRUE))

counties_map <- counties %>%
  left_join(bmp_cty, by = c("GEOID10" = "FIPS"))
```

```r
#Converting the cost to millions for clean look
counties_map$total_cost_m <- counties_map$total_cost / 1e6

# plotting the Map
tm_shape(counties_map) +
  tm_polygons(
    "total_cost_m",
    palette = "Blues",
    style = "quantile",
    title = "Total BMP Cost ($ millions)"
  ) +
  tm_layout(
    main.title = "BMP Investment by County",
    main.title.size = 1,
    main.title.position = "center",
    legend.outside = TRUE,
    outer.margins = c(0.12, 0.02, 0.02, 0.02),
    frame = FALSE,
    bg.color = "white",
    asp = 0
  )
```

```
##
## -- tmap v3 code detected -------------------------------------------------------
## [v3->v4] `tm_polygons()`: instead of `style = "quantile"`, use fill.scale =
## `tm_scale_intervals()`.
## i Migrate the argument(s) 'style', 'palette' (rename to 'values') to
##   'tm_scale_intervals(<HERE>)'[v3->v4] `tm_polygons()`: migrate the argument(s) related to the legend
## visual variable `fill` namely 'title' to 'fill.legend = tm_legend(<HERE>)'[v3->v4] `tm_layout()`: use
## `cols4all::c4a_gui()` to explore them. The old palette name "Blues" is named
## "brewer.blues"Multiple palettes called "blues" found: "brewer.blues", "matplotlib.blues". The first
```

BMP Investment by County




```
#2.4 For each removed dam, find the closest stream segment
# Get nearest stream index
nearest_id <- st_nearest_feature(dams, streams)

# Calculate distance
nearest_dist <- st_distance(dams, streams[nearest_id, ], by_element = TRUE)

# Adding both to dataset
dams_nearest <- dams %>%
  mutate(
    nearest_stream_id = nearest_id,
    nearest_distance_m = as.numeric(nearest_dist)
  )

# Show first 5 rows with the new columns
dams_nearest %>%
  select(nearest_stream_id, nearest_distance_m) %>%
  head()
```

```
## Simple feature collection with 6 features and 2 fields
## Geometry type: POINT
## Dimension:     XY
## Bounding box:  xmin: -78.3887 ymin: 38.4618 xmax: -75.73443 ymax: 41.51775
## Geodetic CRS:  WGS 84
## # A tibble: 6 x 3
##   nearest_stream_id nearest_distance_m            geometry
##               <int>              <dbl>         <POINT [°]>
## 1                53                  0  (-75.73443 38.4618)
## 2              2483                  0  (-78.3887 40.38991)
## 3              1711                  0 (-76.86169 39.86292)
## 4              2282                  0  (-78.2551 41.51775)
```

```
## 5              1758                   0 (-78.32606 41.23462)
## 6              2667                   0 (-77.35531 41.06447)
```

```
st_intersects(dams[1, ], streams[nearest_id[1], ], sparse = FALSE)
```

```
##      [,1]
## [1,] TRUE
```

```
#2.5 Calculate how many removed dams are (or were) in each state

dams %>%
  group_by(STATE) %>%
  summarise(n_dams = n())
```

```
## Simple feature collection with 3 features and 2 fields
## Geometry type: MULTIPOINT
## Dimension:     XY
## Bounding box:  xmin: -79.03091 ymin: 37.23352 xmax: -75.53803 ymax: 41.71974
## Geodetic CRS:  WGS 84
## # A tibble: 3 x 3
##   STATE n_dams                                                      geometry
##   <chr>  <int>                                                <MULTIPOINT [°]>
## 1 MD         2                          ((-75.73443 38.4618), (-76.06189 39.048))
## 2 PA        27 ((-75.53803 41.46384), (-75.90023 41.20816), (-76.05354 41.6335)~
## 3 VA         5 ((-77.41335 37.23352), (-78.89962 38.05994), (-78.89175 38.05944~
```